DYNAMO PAY

Production readiness audit

Developer review / automated testing / UAT evidence

NOT READY FOR UNRESTRICTED LIVE PAYMENTS

NOT READY for unrestricted live customer payments. 56/56 suites (409 reported tests) passed, but seven additional diagnostic tests reproduced nine gaps. Passing diagnostics mean the gaps exist, not that they are fixed.

Prepared 01 November 2026, 12:26 IST

Source 8186e32b70bd4aa90318a490b90d89a373655a9a

Nine reproduced gaps. Current production configuration and acceptance limits are recorded honestly. This is an audit report; runtime defects have not been repaired or deployed.

Read the findings and release gates before onboarding paying customers. Green regression tests are evidence, not blanket production approval.

Scope and verdict

What this repo does

Dynamo Pay creates merchant payment orders, presents hosted UPI checkout, accepts configured provider or notification evidence, posts ledger records and sends signed merchant callbacks. The assumed workload is multiple independent gaming-site merchants with concurrent players on a shared platform. The current one-active-slot-per-receiving-account allocator and single hosted Free instance constrain that workload.

Verdict

NOT READY for unrestricted live customer payments. 56/56 suites (409 reported tests) passed, but seven additional diagnostic tests reproduced nine gaps. Passing diagnostics mean the gaps exist, not that they are fixed.

Baseline

Prepared 01 November 2026, 12:26 IST. Local HEAD and fetched GitHub main: 8186e32b70bd4aa90318a490b90d89a373655a9a. Render current live commit matches. All audit probes used isolated data. No credentials were rotated, no production policies were enabled, no live money was transferred, and no existing financial records were reset.

Skills and review stages

Ponytail, Ponytail Audit and Render Monitor were used. No separate Developer, Tester or UAT skill was available in the session or found in the local skill catalog. Developer review, automated testing and a UAT matrix were performed directly. The PDF skill was used to render and inspect this report.

Coverage honesty

332 tracked source/config/test files were read by the inventory scanner. Critical paths were traced, with section-level review in 48 files; executed suites and review depth are recorded per file in the CSV. Inventory reading, syntax checks and green tests are not semantic sign-off for every line. Legacy TypeScript and auxiliary/demo paths are not all active production code. Runtime code was not changed by this audit.

Current production snapshot

Live service

dynamo-pay, region singapore, plan free, deploy status live, branch main, auto-deploy trigger checksPass. Public origin https://dynamo-pay.onrender.com. Local, remote and live source agree at 8186e32.

Database

PostgreSQL status available, plan free. Current expiry returned by Render: 2026-11-30T06:37:08.293173Z. Arrange migration/renewal before 30 November 2026; no plan change was made. Always-on service, backup/restore evidence and recovery objectives are outstanding.

Current configuration

Read-only aggregate snapshot: 1 merchant record, states [{'status': 'ACTIVE', 'count': '1'}]; 0 scoped API keys; 0 provider connections; 0 Android devices; 0 bound phones; 0 enabled notification policies; 0 payment sessions and 0 callback records. This is not a completed customer installation. Legacy derived credentials are separate from the zero scoped-key count. No inbox credential values were requested or disclosed.

Live HTTP smoke

/_ready = 200; /platform = 200; /developers = 200; /downloads = 200; /api/platform/v1/receiving-mirror = 401; /v1/payments = 401; /api/v1/admin/sms-gateway/download-apk = 503. Anonymous protected APIs correctly return 401. Readiness/portal responses show liveness only. APK 503 blocks current public observer installation. No full authenticated customer payment was executed on production.

Observability limits

The Render connector previously required reauthentication and CLI was unavailable; existing saved API access in the user-confirmed Nitish workspace supplied service/deploy/database metadata. Historical runtime logs and cloud CPU/memory/HTTP latency metrics were not certified in this pass. Browser inventory returned no usable browser and native pipe unavailable; visual UAT cannot be passed.

Verification evidence

Regression

56/56 suites, 409 reported tests, 233.17 seconds; testedAt 2026-11-01T06:53:13.142Z. Some service-dependent acceptance tests may be skipped. The curated runner omits tracked suites: packages/db/src/__tests__/sqlite_verify.test.js, scripts/clean-postgres-history.test.js. Both were executed separately: two tests passed, including PostgreSQL cleanup against its dedicated synthetic schema. The pre-existing untracked e2e-upi-flow.test.js was not promoted into the official run.

Static checks

Typecheck exit 0. Runtime build exit 0: 214 JavaScript runtime/tool files validated (including the untracked benchmark script present locally). Tracked-source secret-pattern scan passed for 571 paths; this is not a full Git-history credential audit.

Dependencies

npm audit --omit=dev reported 0 known production dependency advisories at scan time. A zero advisory count does not certify authentication, financial logic or unknown supply-chain vulnerabilities.

PostgreSQL integration

New separate temporary-schema test passed notification SUCCESS, clearing ledger, outbox, source label, duplicate idempotency, atomic rollback and retry after rollback. Connection search_path was set before adapter construction; public customer rows were untouched and the schema was dropped. These checks are real PostgreSQL queries, not provider/phone acceptance.

Diagnostic probes

Seven isolated diagnostic tests reproduced nine gaps F01-F09. Evidence is in production-audit-probes-2026-11-01.json and the executable diagnostic script. Its assertions deliberately verify existing defects; it is not part of the curated passing production regression suite.

Performance

Earlier 50-sample local on-disk SQLite/HTTP benchmark: alert-to-callback p50 8.36 ms, p95 13.68 ms; wallet p95 14.86 ms. Automatic retry after initial 503 completed in 6.59 seconds. No real bank, Android app, hosted payment latency or customer backend was included.

Security and origin trust

F01 / HIGH / Any app on the paired phone can supply accepted credit-like notifications

apps/android-observer-app/app/src/main/java/com/youpi/observer/PaymentNotificationListenerService.java:30; apps/payment-api/src/routes/pollers.routes.js:348

What this is: The Java listener signs and forwards matching incoming-credit text. It does not check the originating app package/signing identity.

Problem: A signed payload labelled with a test non-bank package and an unused synthetic UTR changed an isolated matching order to SUCCESS. This reproduces the server gap; an actual malicious Android APK was not installed. Pairing authenticates the phone, not the bank.

Fix: Bind permitted source packages and signing identities per receiving account; preserve and verify that provenance in the signed payload. Keep notification acceptance explicitly labelled as a platform policy. Genuine bank verification still requires independent bank/provider evidence.

If skipped: An untrusted local app on the receiving phone can cause an unpaid order to look Paid and trigger game-wallet credit.

F05 / HIGH / Webhook destination checks do not validate resolved IP addresses

packages/common/src/secrets.js:180; packages/core/src/webhook-engine.js:134

What this is: The URL guard blocks private literal IPs and a few private hostname suffixes. Callback delivery uses ordinary fetch DNS resolution.

Problem: With private-webhook permission disabled, http://127.0.0.1.nip.io/callback was accepted, and DNS resolved that hostname to 127.0.0.1. This probe confirmed hostname validation plus resolution; it did not send an exploit request into production.

Fix: Use a shared outbound HTTP client that checks every resolved address and pins the validated destination through connection establishment, for webhook and integration requests. Continue refusing redirects and private/reserved networks.

If skipped: A merchant-controlled DNS name can route callback requests into local/private services: server-side request forgery.

Refund and callback correctness

F03 / HIGH / SUCCESS can commit without a callback outbox record

apps/payment-api/src/services/receiving-mirror.js:78; packages/core/src/webhook-engine.js:45

What this is: enqueue returns an empty delivery ID when the merchant URL is missing or rejected. The mirror does not treat this return value as a reservation failure.

Problem: Removing the callback URL after setup, then accepting an alert, produced SUCCESS and a ledger entry with zero webhook_deliveries rows. Restoring the URL later has no row to retry.

Fix: Persist a durable pending callback event even when the endpoint is temporarily unconfigured, or reject/pause acceptance until a callback reservation can be made. Alert operators and preserve an explicit replay path.

If skipped: The game never receives its success event although the customer has paid and the portal shows Paid.

F06 / HIGH / Refund Idempotency-Key does not prevent a second refund

apps/payment-api/src/platform/payments.js:193; apps/payment-api/src/routes/refunds.routes.js:61

What this is: The scoped refund wrapper requires Idempotency-Key but does not bind it to a refund request. Without body.refundId, the delegated route generates a new random refund ID each time.

Problem: Two identical authenticated /v1/refunds requests with the same Idempotency-Key and amount created two refund rows and called the simulated provider twice. Both fit within the remaining original balance, so the cumulative reservation guard did not prevent the duplicate intention.

Fix: Atomically bind merchant plus idempotency key to a canonical refund fingerprint and stable refund ID; reject changed requests and return the original result for a retry. Test timeout, duplicate and partial-refund cases through /v1/refunds.

If skipped: A normal merchant network retry can refund a customer twice, up to the remaining refundable amount.

Receiving alert correctness

F02 / HIGH / Native notification title is lost during server parsing

apps/payment-api/src/routes/pollers.routes.js:325; apps/android-observer-app/app/src/main/java/com/youpi/observer/ApiClient.java:169

What this is: Native extraction reads title plus text and sends both inside rawNotification. The paired server path reparses rawNotification.text alone.

Problem: A native-shaped payload with amount/received in the title and UTR in the text returned HTTP 200 NON_PAYMENT_ALERT. The matching order remained ACTIVE even though the native payload also supplied parsed amount/reference.

Fix: Normalize title and text together before the trusted server parser, with a bounded combined length. Test actual Slice/GPay/PhonePe notification formats and duplicate expanded/collapsed notifications.

If skipped: Valid receiving notifications can be silently acknowledged and removed from the phone queue without marking the order Paid.

F04 / HIGH / A one-to-two-minute-late alert can miss the pending order

apps/payment-api/src/services/receiving-mirror.js:51; packages/core/src/timing-engine.js:156

What this is: Processing timeout moves an order to VERIFICATION_PENDING, but mirror eligibility still cuts off at processing_expires_at.

Problem: An alert observed two minutes after order creation, after its 30-second processing window, returned UNMATCHED while the order stayed VERIFICATION_PENDING. The ten-minute creation cap does not extend the earlier processing expiry.

Fix: Define a safe late-confirmation window and correlate by a real order-bound reference or authoritative evidence. Do not simply relax the amount/time match when the same UPI can have a new same-amount order.

If skipped: A valid late bank/app notification will not generate the immediate SUCCESS callback the product promises for delayed payments.

Legacy customer integration

F07 / MEDIUM / Legacy collection still has a hard INR 100 minimum

apps/payment-api/src/routes/dynamopay.routes.js:191

What this is: The preferred /v1/payments accepts positive paise under saved rules. The still-supported /v1.0/collect/upi route has an additional hard minimum.

Problem: A legacy request for INR 1 was rejected INVALID_AMOUNT even with channel minimum saved as 0. This contradicts the requested zero configurable floor across integrations.

Fix: Reuse the canonical positive-paise and saved-channel validation in the legacy adapter; add a below-INR-100 legacy regression case.

If skipped: Customers using the legacy SDK/API cannot accept the same small payments that the preferred API allows.

F08 / MEDIUM / New merchant legacy credentials fail until server restart

apps/payment-api/src/routes/dynamopay.routes.js:91

What this is: Legacy merchant credentials are loaded into a process-local map only when routes are registered.

Problem: A merchant created after registration received HTTP 401 from the legacy wallet endpoint even with correctly derived legacy credentials. The scoped dp_live API uses the database and is not this cache.

Fix: Resolve the current merchant and derived credential on demand, or refresh/invalidate the legacy cache when onboarding changes it. Test onboarding then immediate legacy API use without restarting.

If skipped: An onboarded customer can receive valid legacy details that do not work until the service is restarted.

F09 / MEDIUM / Legacy wallet omits notification-confirmed funds

apps/payment-api/src/routes/dynamopay.routes.js:145; packages/db/src/repositories/ledger.repository.js:21

What this is: Notification collections correctly post into NOTIFICATION_CLEARING. The legacy wallet unsettled projection only reads PROVIDER_CLEARING.

Problem: After an isolated INR 100 notification-confirmed payment, NOTIFICATION_CLEARING held 10,000 paise while /v1.0/wallet returned unsettledbalance 0.00 and no separate notification-clearing field.

Fix: Expose notification-clearing funds as a separately named accounting balance and reconcile it to actual bank credit. Do not label it settled bank cash.

If skipped: The customer sees successful deposits but incomplete recorded balance information and may reconcile against the wrong total.

UAT checklist / 1

NOT RUN / BLOCKED is not a pass. FAIL IN ISOLATED PROBE is a confirmed code gap, not a live exploit or bank test. See CSV for complete evidence and responsible actor.

ID	Acceptance case	Status
U01	Sign in and verify MFA	BLOCKED
U02	Create invite, deliver email/mobile, accept and submit profile	NOT RUN
U03	Approve and activate intended merchant with correct route	NOT RUN
U04	Issue merchant-scoped key, store secret, revoke test key	NOT RUN
U05	Assign receiving UPI and acceptance policy	NOT RUN
U06	Build/sign APK and scan one-time QR	BLOCKED
U07	Reject notifications from an unrelated installed app	FAIL IN ISOLATED PROBE
U08	Accept title/body split incoming alert	FAIL IN ISOLATED PROBE
U09	Accept permitted real Slice/GPay/PhonePe credit with UTR	NOT RUN
U10	Create payment, render checkout/QR, read Paid status	NOT RUN ON HOSTED MERCHANT
U11	Verify raw signature and credit wallet exactly once	NOT RUN WITH CUSTOMER
U12	Return 503 once, then receive automatic retry	NOT RUN WITH CUSTOMER
U13	Same refund key retry creates only one refund	FAIL IN ISOLATED PROBE

UAT checklist / 2

NOT RUN / BLOCKED is not a pass. FAIL IN ISOLATED PROBE is a confirmed code gap, not a live exploit or bank test. See CSV for complete evidence and responsible actor.

ID	Acceptance case	Status
U14	Callback URL removal does not lose success delivery	FAIL IN ISOLATED PROBE
U15	One-to-two-minute-late credit reaches pending order safely	FAIL IN ISOLATED PROBE
U16	Reject hostname resolving to private callback destination	FAIL IN ISOLATED PROBE
U17	Legacy API accepts low amount under zero-minimum policy	FAIL IN ISOLATED PROBE
U18	New merchant legacy API works without restarting	FAIL IN ISOLATED PROBE
U19	Notification-clearing amount appears separately in balance	FAIL IN ISOLATED PROBE
U20	Shared UPI, identical amount, overlapping orders	NOT RUN WITH CUSTOMERS
U21	Offline queue, reconnect, revoked phone	NOT RUN ON DEVICE
U22	OAuth consent, inbox binding, real credit email	NOT RUN
U23	Provider collection/refund/payout and settlement reconciliation	NOT RUN
U24	Restore backup and restart with outstanding callback	NOT RUN IN HOSTED RECOVERY
U25	Desktop/mobile form, navigation, accessibility and errors	BLOCKED
U26	Measure actual bank-to-phone-to-host-to-wallet p50/p95	NOT RUN

Module and file coverage

How to read the inventory

332 source/config/test files are listed one by one, including module, line count, review depth, finding IDs and SHA-256 hash. Critical paths received section/caller review; remaining files retain inventory-only status. This is not a false claim that every line of legacy, native, TypeScript, demo and dependency code has been semantically audited.

Modules

apps/android-bridge (16); apps/android-observer-app (22); apps/checkout-portal (10); apps/coin-wallet-store (4); apps/control-center (5); apps/gmail-observer (7); apps/merchant-admin (11); apps/payment-api (66); apps/payment-help (4); apps/sms-observer (7); apps/super-admin (3); packages/common (33); packages/core (34); packages/db (51); packages/merchant-sdk (8); packages/observer-sdk (4); scripts/apply-identity-schema.js (1); scripts/audit-db-schema.js (1); scripts/backup-and-reset-customer-db.js (1); scripts/build-android-apk.js (1); scripts/chaos-concurrency.test.js (1); scripts/check-chrome.js (1); scripts/check-runtime.js (1); scripts/clean-postgres-history.js (1); scripts/clean-postgres-history.test.js (1); scripts/clean-transaction-history.js (1); scripts/clean-transaction-history.test.js (1); scripts/clean.js (1); scripts/create-initiated-session.js (1); scripts/create-test-session.js (1); scripts/dev.js (1); scripts/execute-clean-customer-reset.js (1); scripts/full-regression-e2e.js (1); scripts/health-check.js (1); scripts/hosted-routing.test.js (1); scripts/load-test-upi-mirror.js (1); scripts/migrate-to-postgres.js (1); scripts/postgres-acceptance.test.js (1); scripts/postgres-bootstrap.js (1); scripts/production-infrastructure.test.js (1); scripts/recovered-chunks.json (1); scripts/reset-production-db.js (1); scripts/scan-secrets.js (1); scripts/scratch-check-db.js (1); scripts/serve-for-load.js (1); scripts/start-hosted.js (1); scripts/start-local.js (1); scripts/start-quick-tunnel.js (1); scripts/start.js (1); scripts/sync-checkout-html.js (1); scripts/test-all.js (1); scripts/test-cdp.js (1); scripts/test-support.js (1); scripts/validate-env.js (1); scripts/verify-53-point-auth-matrix.js (1); scripts/verify-all-prod-extensions.js (1); scripts/verify-db-isolation.js (1); scripts/verify-modes.js (1); scripts/verify-prod-checkout.js (1); scripts/verify-prod-upi.js (1); scripts/verify-spawn.js (1); scripts/verify-support-and-upi-extensions.js (1); scripts/webhook-worker.js (1)

What tests prove

The ordinary suite shows expected behavior on its scenarios. The additional diagnostic suite proves the nine counterexamples. PostgreSQL acceptance validates isolated data behavior on the real engine. None of those proves bank settlement, customer wallet correctness or native installation on the receiving phone.

Artifacts

PDF and editable Markdown report; file-level coverage CSV; 26-case UAT CSV; source diagnostic test; hosting/database/HTTP snapshots; regression result and logs; dependency JSON; diagnostic results; PostgreSQL acceptance result; earlier timing benchmark. No real credentials, backups, private PDFs or connection strings are included.

Required production release gates

Fix order

First repair F06 refund idempotency and F05 DNS-safe egress; lock down F01 source provenance before using notification-based game-wallet credit. Then repair F02 title/body normalization, F03 callback reservation and F04 late-confirmation design. Resolve F07-F09 before supporting legacy integrations.

Release verification

Turn the reproduced cases into normal regression tests expecting the corrected behavior. Run typecheck, runtime build, secret/dependency checks, the 56-suite regression and the isolated PostgreSQL acceptance again. Review exact commit/CI/deployment; do not treat these still-failing behaviors as approved merely because the old tests pass.

Infrastructure

Choose always-on hosting with measured capacity and an appropriate database retention plan. Set monitoring, external error alerts, backup/restore procedure and recovery ownership. One active receiving slot, shared-UPI amount collisions and sequential callback-retry backlog must be covered by a load/capacity plan.

Customer acceptance

Complete all UAT rows, including signed current APK on Vivo, trusted bank/payment app capture, real small credit and bank statement match, customer raw-signature verification and durable wallet credit once, callback failure/recovery, revoked/offline phone and late alerts. Confirm provider/bank acceptance separately for refunds, payouts and external collections.

Callback timing limits

Immediate callback attempt happens after accepted evidence commits. Default background callback sweep is 5 seconds, each HTTP attempt timeout 10 seconds, total default attempts 5. Retry delays used are 5 seconds, 30 seconds, 2 minutes and 10 minutes. The retry loop awaits rows sequentially; 20 unresponsive endpoints can occupy a sweep for roughly 200 seconds, although online alert-specific immediate attempts are separate. This is a code-derived worst-case scenario, not a measured hosted load test.

Final sign-off

Verdict: NOT READY. A controlled non-money demonstration can show the mechanics, but unrestricted customer collections and an instant-callback SLA are not justified by current evidence. Not checked: every historical line/dependency manually, native compilation/device, genuine bank/provider flows, customer game-wallet integration, browser visual UAT, hosted high-load/recovery tests and production CPU/memory/log metrics. No automatic repair, commit, merge or deployment was performed in this audit.